Informatik I **Wintersemester 2025/26**
Dr. Henrik Brosenne Georg-August-Universität Göttingen
Paul Sarnighausen-Cahn Institut für Informatik

Übungsblatt 04

Allgemeine Information zum Bestehen der Prüfungsvorleistung.
Um die Prüfungsvorleistung zu bestehen, müssen Sie auf jedem Übungszettel mindestens 40% der Punkte in Theorie und Praxis erreichen. Es gibt jedoch zwei Ausnahmen: Sie dürfen auf zwei Zetteln bei den theoretischen Aufgaben und auf zwei Zetteln beim praktischen Teil unter die 40%-Grenze fallen, ohne dass dies zum Nichtbestehen führt.
Das bedeutet konkret: Wenn Sie beispielsweise auf einem Übungsblatt weniger als 40 % in der Theorie erreichen, können Sie den praktischen Teil trotzdem bestehen – und umgekehrt.

Theoretische Übungen

Abgabe bis Di., 09.12., 14 Uhr

Allgemein Die Lösung dieser Aufgaben werden über das StudIp-Modul Vips abgegeben.

Achten Sie darauf, dass Ihre Abgabe gut lesbar ist.

Formatieren Sie Ihre Abgabe so, dass die Aufgaben 1, 2 und 3 klar durch einzelne Überschriften voneinander getrennt sind.

Aufgabe 1 – 25 Punkte

8-Bit Gleitkommazahlen (i1-u-fl0)

Wir betrachten eine vereinfachte Variante des IEEE-754-Standards das in der Vorlesung vorgestellt wurde. Hier betrachten wir **8-Bit Gleitkommazahlen** (Minifloats). Diese bestehen aus:

7	6	5	4	3	2	1	0
S	E				T		

Interpretation:

E	T	Wert	Typ
$1 \leq E \leq 14$	beliebig	$(-1)^S \cdot 2^{E-7} \cdot (1 + 2^{-3} \cdot T)$	normal
$E = 0$	$T \neq 0$	$(-1)^S \cdot 2^{1-7} \cdot (0 + 2^{-3} \cdot T)$	subnormal
$E = 0$	$T = 0$	$(-1)^S \cdot 0$	zero
$E = 15$	$T \neq 0$	NaN	not a number
$E = 15$	$T = 0$	$(-1)^S \cdot \infty$	infinity

Aufgabe
Berechnen Sie **von Hand** den numerischen Wert der folgenden 8-Bit Gleitkommazahlen (zur Basis 2 nach obiger Definition). Geben Sie **alle Rechenschritte** an (S, E, T interpretieren, Typ bestimmen, Wert berechnen).

1. `0 1001 011`
2. `1 0111 100`
3. `0 0000 101`
4. `1 1111 000`

(20 Punkte) **Hinweise:**

- `S` ist das Vorzeichenbit.
- `E` ist als 4-Bit-Zahl zu interpretieren, `T` als 3-Bit-Zahl.
- Rechnen Sie zuerst S, dann $E - 7$, dann den Mantissenwert.
- Verwenden Sie zum Rechnen Potenzen von 2 (keinen Taschenrechner).

Welches ist der größte positive darstellbare Wert und welcher der kleinste positive darstellbare Wert (nicht 0) in diesem System? Begründen Sie Ihre Antwort.
(5 Punkte)

Aufgabe 2 – 10 Punkte

Betrachten Sie Zeichenebene, Codierung.

- Betrachten Sie folgenden Python-Code:

```
a = "test"
b = a
a = a + "!"
c = a
a = "te" + "st!"
```

  1. Warum ändert sich der Wert von `b` nicht, obwohl `a` verändert wurde?
  2. Wie viele verschiedene Objekte existieren am Ende im Speicher?

(10 Punkte)

Aufgabe 3 – 15 Punkte

Boolesche Ausdrücke und Auswertungsstrategie (i1-u-bo0)

Gegeben sind die booleschen Ausdrücke:

$$\text{(A)} \quad (x \wedge \neg y) \ \vee \ (\neg x \wedge y) \qquad \text{(B)} \quad (x \vee y) \ \wedge \ \neg(x \wedge y)$$

1. Erstellen Sie eine vollständige Wahrheitswerttabelle für A und B.
2. Prüfen Sie, ob A und B logisch äquivalent sind.
3. Geben Sie eine äquivalente Python-Formulierung an.

(10 Punkte)

- **Evaluation:** Betrachten Sie folgenden Python-Code:

```
x = False
y = True
print(x and (y or print("Hallo")))
print(x or (y and print("Welt")))
```

1. Welche Ausgabe erscheint auf der Konsole, und warum?

Hinweis:

- Um zu testen welchen Wahrheitswert ein Funktionsaufruf hat, können Sie die Python Funktion *bool*() ausprobieren.

(5 Punkte)

Praktische Übung

Abgabe der Prüfsumme bis Di., 09.12., 14 Uhr
Testat ab Di., 09.12., 18 Uhr

Hilfe zum Bearbeiten der praktischen Übungen können Sie grundsätzlich jeden Tag in den Rechnerübungen bekommen.

- Bearbeiten Sie die praktischen Übungen in dem bereitgestellten Jupyter Notebook.
- Um die praktische Übung testieren zu lassen, **müssen** Sie einen Termin in der zugehörigen Rechnerübung reservieren. Ein Testat ohne Termin ist nicht möglich.
- Übermitteln Sie die Prüfsumme über Vips
- Öffnen Sie die Terminvergabe für diese praktische Übung und lassen Sie den von Ihnen reservierten Termin anzeigen.
- Zeigen Sie ihr Notebook und führen Sie alle zellen aus, damit am Ende die Prüfsumme angezeigt wird.
- Der CIP-Pool befindet sich hier: `https://lageplan.uni-goettingen.de/?ident=2412_1_1.UG_-1.110`

Aufgabe 1 – 15 Punkte

For-Schleife, Boolsche Ausdrücke, If-Else (i1-p-wn0)

Sie haben Ihren Lieblings-Integer vergessen, aber erinnern sich an folgende Eigenschaften:

- Die Zahl liegt irgendwo zwischen 1 und 1000.
- Die Zahl ist ohne Rest durch 6 teilbar.
- Die Zahl ist ohne Rest durch 7 teilbar.
- Die Zahl ist nicht ohne Rest durch 4 teilbar.
- Die Quersumme der Zahl ist 6.

Schreiben Sie eine kleine Python-Simulation um herauszufinden, welches Ihre Lieblingszahl war.
Hinweis
Sie können schon geschriebene Funktionen, wie z.B. die Quersumme wiederbenutzen.
(15 Punkte)

Aufgabe 2 – 35 Punkte

Hase-Igel-Algorithmus (i1-p-ig0)

Zu *Folge mit einfachem rekursivem Bildungsgesetz* und *Mittelquadratmethode* siehe Praktische Übung 03.

Mit dem *Hase-Igel-Algorithmus* kann in einer Folge mit rekursiver Bildungsvorschrift ein Zyklus gefunden werden.

- Die Folge wird mit zwei Variablen durchlaufen: **Hase** und **Igel**.
- In jeder Iteration bewegt sich der Hase zwei Folgenglieder weiter, der Igel eines.
- Treffen Hase und Igel auf denselben Wert, wurde ein Zyklus gefunden. Ein Zyklus ist länger als 1, wenn das nächste Folgenglied des Igels nicht gleich dem aktuellen ist.

1. Programmieren Sie in Python eine Funktion, die einen *Middle-Square-Generator* für **4-stellige Hexadezimalzahlen (16 Bit)** realisiert.

   Hinweise:

   - Ein- und Ausgabe kann dezimal erfolgen.
   - Python-`int` kann beliebig große Zahlen speichern, d. h. es ist kein spezieller Datentyp nötig. Bei der Umrechnung können aber Hexadezimalstrings (z. B. `"0x1234"`) sowie `int(..., 16)` hilfreich sein.
   - Um nach dem Quadrieren wieder auf 4 Hexadezimalstellen zu kürzen, kann z. B. folgender Ausdruck verwendet werden:

     ```
     x = (x * x) // 0x100 % 0x10000
     ```

     Dabei entspricht `0x10000 = 2**16`.

   Schreiben Sie eine Funktion:

   ```
   def middle_square(x):
       ...
       return next_value
   ```

   und testen Sie diese für einige Startwerte.

2. Erweitern Sie Ihr Programm aus Teil 1. Verwenden Sie den *Hase-Igel-Algorithmus*, um alle 4-stelligen Hexadezimalzahlen im Bereich von `0` bis `0xFFFF` (= 65535) als Startwert zu testen, und finden Sie alle Startwerte, die zu einem Zyklus der Länge größer 1 führen.

   - Verwenden Sie zwei Variablen, z. B. `hase` und `igel`, die ausgehend vom gleichen Startwert iterativ aktualisiert werden.
   - Bewegen Sie den Hase pro Schleifendurchlauf zweimal weiter, den Igel einmal.
   - Stoppen Sie, sobald beide denselben Wert haben.
   - Kommentieren Sie wesentliche Schritte (Berechnung, Aktualisierung von Hase und Igel).

Hinweis:
Da es nur `0x10000` mögliche Werte gibt, muss spätestens innerhalb der ersten `0x10000` Schritte ein Werte-Duplikat auftreten, also ein Zyklus existieren.

Geben Sie alle Startwerte aus, die zu einem Zyklus der Länge größer 1 führen.

(35 Punkte)